1. Overview

- **Project Name:** Luma ERP
- **Purpose:** ERP system for apparel manufacturing.
- **Tech Stack:** Django, PostgreSQL, AWS S3, JavaScript, Bootstrap.
- **Architecture:** Modular client-server model with key modules: UserProfiles, Models, Orders, Clients, Works.

2. Setup Instructions

- **Prerequisites:** Python 3.10.10, JavaScript, PostgreSQL.
- **Installation:**

```
git clone https://github.com/TemirlanSadykov/AktisERP
python -m venv venv
Mac/Linux: source venv/bin/activate
Windows: .\venv\Scripts\activate
cd AktisERP
pip install -r requirements.txt
git checkout merps-test
```

- **Environment Variables:** Create a `.env` file with the following credentials:

```
AWS_ACCESS_KEY_ID=AKIA5FTZEHCYA6UXLK5Q
AWS_SECRET_ACCESS_KEY=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
AWS_STORAGE_BUCKET_NAME=aktiserpbucket
AWS_S3_REGION_NAME=ap-southeast-1
MERPS_TEST_DB_ENGINE=django.db.backends.postgresql
MERPS_TEST_DB_NAME=railway
MERPS_TEST_DB_USER=postgres
MERPS_TEST_DB_PASSWORD=gRkLSPUUIQZkKzybTNnhrbANYEyHrBDe
MERPS_TEST_DB_HOST=junction.proxy.rlwy.net
MERPS_TEST_DB_PORT=55307
```

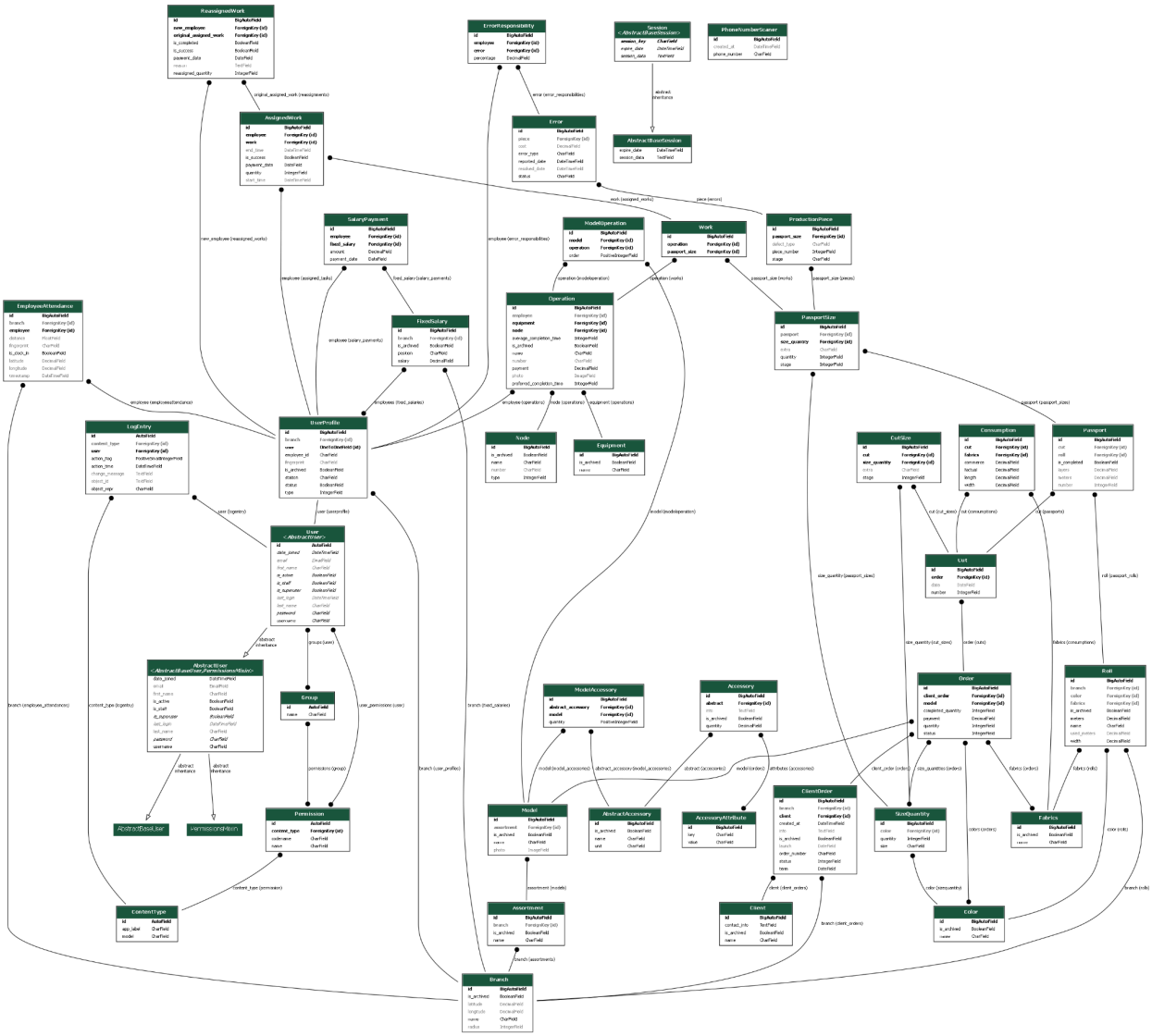
- **Run App:**

```
python manage.py runserver
```

3. Project Structure

```
AktisERP/
├── aktis/
│   ├── .env                  # Environment variables file
│   ├── .git                  # Git repository data
│   ├── .gitignore            # Specifies files to be ignored by Git
│   ├── aktis/
│   │   ├── __init__.py       # Init file for project package
│   │   ├── asgi.py           # ASGI configuration for async support
│   │   ├── settings.py       # Main settings file for the project
│   │   ├── urls.py           # Root URL configurations
│   │   ├── wsgi.py           # WSGI configuration for deployment
│   │   └── __pycache__/      # Compiled Python files
│   ├── django_debug.log      # Log file for debugging
│   ├── locale/
│   │   ├── bn/               # Bengali translations
│   │   └── ru/               # Russian translations
│   ├── manage.py             # Django's command-line utility
│   ├── Procfile              # Deployment configuration (e.g., for Railway)
│   ├── production/
│   │   ├── __init__.py       # Init file for app package
│   │   ├── admin.py          # Admin configuration for models
│   │   ├── apps.py           # App registration and configuration
│   │   ├── decorators.py     # Custom view decorators
│   │   ├── forms.py          # Custom forms for user input
│   │   ├── middleware.py     # Custom middleware classes
│   │   ├── migrations/       # Database migration files
│   │   ├── mixins.py         # Custom view mixins
│   │   ├── models.py         # Database models
│   │   ├── storage_backends.py # Custom storage configurations (e.g., AWS S3)
│   │   ├── templates/        # HTML templates for the app
│   │   ├── templatetags/     # Custom template tags and filters
│   │   ├── tests.py          # Test cases for the app
│   │   ├── urls.py           # URL configurations specific to the app
│   │   ├── views/
│   │   │   ├── admin_views.py # Views for admin users
│   │   │   ├── employee_views.py # Views for employees
│   │   │   └── ...           # Additional views organized by role or feature
│   │   ├── views.py          # General views
│   │   ├── management/
│   │   │   └── commands/
│   │   │       └── populate_dummy_data.py # Custom management command
│   │   └── tasks.py          # Automated tasks or scheduled jobs
│   ├── requirements.txt      # Project dependencies
│   ├── runtime.txt           # Python version for deployment
│   ├── static/
│   │   ├── css/              # CSS files
│   │   ├── favicon.ico       # Favicon for the application
│   │   ├── fonts/            # Fonts used in the application
│   │   ├── img/              # Image assets
│   │   └── js/               # JavaScript files
│   ├── staticfiles/          # Static files collected for production use
│   └── translations/         # Additional translation files (currently empty)
└── venv/                     # Virtual environment for dependencies (not in version control)
```

4. Main Components and Features

- **Core Features:**
  - User authentication and role management
  - Inventory tracking and materials management
  - Production tracking, including barcode generation
  - Order management and client tracking
  - Analytics dashboard with production and inventory metrics
- **Key Models:**
  - `UserProfile`: Stores user data and roles (Admin, Technologist, etc.)
  - `ProductionPiece`: Tracks individual production units with status updates
  - `Client Order, Order, Cut, Passport`: Records client orders, linked to production pieces
  - `Roll` & `Accessory`: Track material usage and production details
- **Dashboard APIs:**
  - `/api/clients/<int:client_id>/` - Retrieve details for a specific client
  - `/api/employees/<int:employee_id>/` - Retrieve details for a specific employee
  - `/api/orders/<int:order_id>/` - Retrieve details for a specific order
  - `/api/rolls/<int:roll_id>/` - Retrieve details for a specific roll
  - `/api/client-orders/<int:client_order_id>/` - Retrieve details for a specific client order
  - `/api/fetch-model-records/` - Fetch records for models
  - `/api/fetch-record-details/` - Fetch detailed information on specific records

5. Database Schema